Seat 1 Build Plan: Credit Analyst

Oct 24, 2026 · @ary

Goal and scope

By Saturday 18:00, the Credit Analyst agent runs inside Rajeev's repo. It writes a `credit_profile` row for all 30 clients and raises seven `exploit_flag` rows. It also proposes tier changes as `CREDIT_REVIEW` work items, which only a controller can approve.

Seat line for Devpost: Fills the Credit Analyst seat at an institutional crypto platform. It scores every client on reliability, value and coverage, and predicts when they will pay. It moves clients between the platform's five credit tiers, and it catches clients gaming the payment terms, with evidence. Every tier change goes to a controller.

Seat 1 builds	Seat 1 never does
Reliability, value and coverage scores, plus an expected pay date, per client	Restrict a lever, hold cash or debit custody (that is Seat 2)
Six exploit detectors with evidence rows	Approve its own proposals
Tier change proposals with reasons	Change a strategic client's tier
An audit event for every write	Edit contract grace or cure days mid-contract
Tests against Rajeev's planted cases	Let the model pick a score or a tier

The scoring is deterministic Python. Claude only writes the plain-English reasoning on each proposal, with a template fallback.

What Rajeev's repo already gives us

Most of what Seat 1 needs already exists. Three things are missing: a reducer to write `credit_profile`, a reducer to write `exploit_flag`, and code that applies an approved `CREDIT_REVIEW`. Checked against commit `793e1bd` on projects-hacks/access-desk.

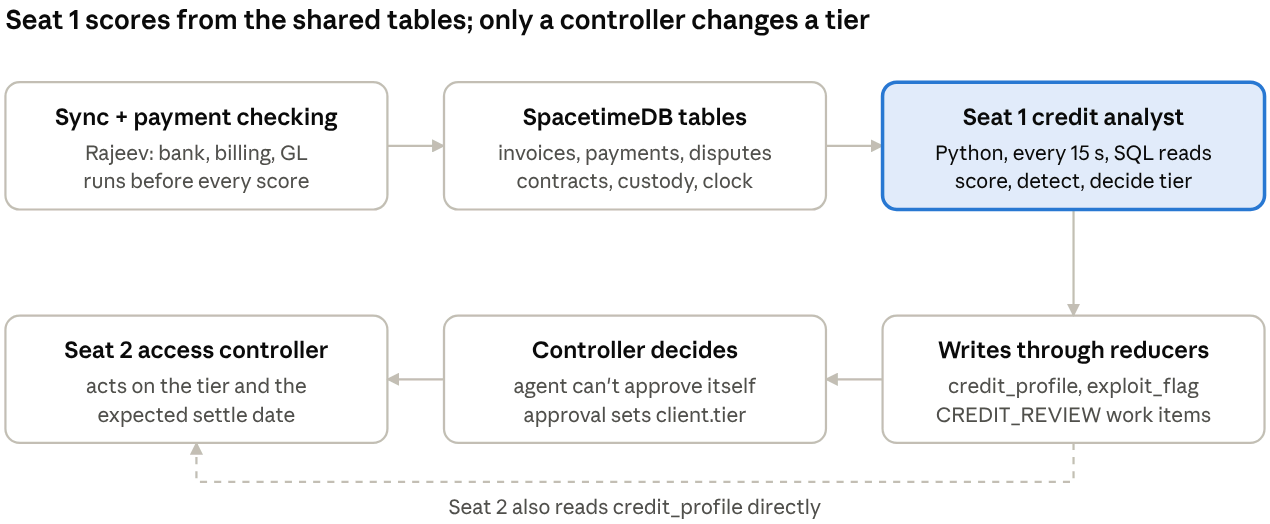
Seat 1 needs	Where it is in the repo	State
Client list, tier 1–5, strategic flag	`client` table (`tier` u8, `strategic` bool)	Ready
Grace, cure, cross-default per client	`contract` table; `contractFor()` sets grace 15/10/10/5/3 and cure 15/15/15/10/10 by tier	Ready
A year of paid history	`invoice` (`paid_on`, `paid_cents`), `payment_application` → `cash_movement.value_on`, history Mar 2025 – Jan 2026	Ready
Disputes and outcomes	`dispute` (`resolved_on`, `resolution`: `no_change` or `credited`)	Ready
Collateral	`custody_holding`, `owed_to_client`, `contract_clause` (§6.3 debit, §7.2 set-off)	Ready
Output: scores	`credit_profile` (`value_score`, `reliability_score`, `coverage_ratio`, `coverage_detail`, `expected_settle_on`)	Table exists, **no reducer writes it**
Output: exploit flags	`exploit_flag` (`pattern`, `score`, `status`, `months`, `evidence_json`, `proposed_fix`)	Table exists, **only the module writes it**
Output: tier proposals	`propose_work_item` with kind `CREDIT_REVIEW`; `transition_work_item` blocks self-approval	Proposing works, **approval does nothing yet**
Agent runtime	`agents/` Python package: `SpacetimeClient.sql()`, `.call()`, `.propose()`; loop every 15 s in `__main__.py`	Ready; `credit_analyst.py` is a stub
Planted cases	`PATTERNS` in `server/src/dataset/history.ts`, `CASES` in `scenario.ts`	Ready

Seat 1 needs	Where it is in the repo	State
Policies to cite	`policies.ts` baseline pol-001 to pol-015	No credit policy yet; add pol-020 to pol-022

Two naming notes. Rajeev's `client_credit` table means unapplied cash, not credit terms, so Seat 1 never writes it. The simulated today is 2026-03-03, read from `clock.sim_date`, never from the wall clock.

Architecture and data flow

Seat 1 reads the shared database with SQL and writes only through reducers, the same rule Rajeev's agents already follow. Nothing it writes changes access until a controller approves.



Seat 1 data flow · 6 parts

Each run, every 15 seconds in `__main__.py`, loads all 30 clients, scores them, writes profiles and flags, and returns only proposals that aren't already open. A new payment or dispute shows up on the next run, so a live judge scenario needs no extra trigger.

Scoring spec

Each client gets three scores and one prediction, written to its `credit_profile` row. Reliability sets the tier, value can move it one step, and coverage never moves it. These numbers are calibrated on Rajeev's 30 clients, and every rule below has been run on his

dataset.

Which invoices count. Every invoice past its due date, excluding void ones and ones under an open dispute. Paid invoice: lag = `paid_on` − `due_on`. Still open: lag = today − `due_on`. Days past due (dpd) = max(0, lag). Weight = 0.5 ^ (age in days / 90), so a 90-day-old invoice counts half.

Reliability (0–100): will they pay on time? Start at 100 and subtract:

Penalty	Rule	Cap
Lateness	0.8 × weighted mean dpd	35
Frequency	20 × weighted share of invoices with dpd > 10	20
Severity	15 if the worst dpd in the last 180 days is over 60; 8 if over 30	15
Paid in pieces	4 × weighted count of invoices settled by 2+ cash movements	10
Disputes lost	1.5 per dispute resolved `no_change`; `credited` costs nothing	15

Then shrink toward a neutral 70 for short histories: reliability = (m × raw + 3 × 70) / (m + 3), where m = billing months scored. Every client in this book has 10+ months, so this only matters for new clients.

Value (0–100): how much do we lose if we upset them? 75 × the client's log of fees billed in the last 12 months, scaled from the smallest client (0) to the largest (1), plus 25 × products used out of 4. The log stops Redwing's $7.2M from flattening Larkspur's $53k.

Coverage ratio: can we collect without them? (USD custody cash, only if the §6.3 debit clause is signed, + `owed_to_client`) ÷ undisputed unpaid amount. `coverage_detail` also records custody value that §7.2 set-off lets Seat 2 hold, with its multiple of exposure. Example: Kestrel's ratio is 0 because it never signed §6.3, but its holdable custody is about 42× what it owes.

Expected settle date: when will they pay? The client's recency-weighted median lag on paid invoices, added to the due date of its oldest open invoice. If that date has passed, use today. Juniper's lag is 20 days, Kestrel's lately 9. Seat 2 can send a reminder instead of restricting when the expected date falls before the cure period ends.

Store scores as integers 0–100 (`value_score` and `reliability_score` are u8). Keep the unrounded breakdown in the work item's `proposal_json` so the reasoning can quote it line by line.

Exploit detectors

Six detectors, one per pattern Rajeev planted in `history.ts`. On his data each fires on its planted client and on no other client, with one exception: recent_late also catches Fieldstone's real slide once its catch-up wire is applied. Every detector returns its measured number even when it doesn't fire, so the reasoning can say why a client was cleared.

Pattern (`exploit_flag.pattern`)	Client	Fires when	Evidence written	Proposed fix
`restriction_eve`	Juniper Labs	85%+ of single-payment invoices in the last 8 months were paid on the exact same day after due, and that day is past grace	14 of 14 invoices paid on day 20	Auto Pay or prepay for API on the due date; shorter cure at renewal
`partial_reset`	Wrenfield Digital	2+ invoices where the first payment was under 60% and the balance landed after the restriction day (grace + 1 + cure)	11 invoices: half at day 8, rest at day 33 vs restriction day 16	Clause: a partial payment reduces the balance but doesn't stop the cure clock

Pattern (`exploit_flag.pattern`)	Client	Fires when	Evidence written	Proposed fix
`pays_restricted_only`	Monarch Rails	The median lag per product differs by 15+ days and the slow one is past grace	API at day 2, custody at day 30	Cross-default at renewal, so a late custody bill puts API in default too
`quarter_end`	Halcyon Capital	5 or fewer payment dates in 12 months, at least 2 months of invoices per date	5 wires for 11 months of invoices	Strategic: controller review only, no automatic fix
`recent_late`	Kestrel Quant	2 of the last 3 months more than 5 days late, and their average is 7+ days worse than before	12, 18 and 9 days late vs about 0 before	No upgrade; recheck at renewal on 2026-03-12
`dispute_delay`	Orchard Treasury	Half or more of months disputed, 80%+ of disputes upheld `no_change`, and 80%+ paid within 3 days of the ruling	11 of 11 months disputed, 22 of 22 upheld	Escalate; disputes filed after a default notice don't stop the cure period

Months are counted by the invoice's `period`. Store `score` as a 0–1 confidence (share of the evidence that matches) and `months` as the months covered. Set `status` to `flagged`, and to `watch` for a strategic client. Each flag id is `<pattern>:<client_id>`, so reruns update the same row instead of adding new ones.

Kestrel and Fieldstone show why `recent_late` is a trend rule, not a lateness rule. Ivory Lane had one 41-day miss in November and is correctly not flagged.

Tier rules, guardrails and approvals

The engine uses Rajeev's five tiers (1 best, 5 worst) and never invents new ones. It computes a formula tier, applies guardrails, and proposes any difference as a `CREDIT_REVIEW` that needs a controller.

Formula tier

Reliability	Base tier
85 or more	2
70 to 85	3
55 to 70	4
under 55	5

A value of 75 or more improves the base by one tier. Tier 1 also needs reliability of 85 or more. Any exploit flag caps the result at tier 3.

Guardrails, applied in this order

1. Strategic client: never propose a tier change. If the formula tier is worse, propose a `review` item instead (Halcyon).
2. Upgrade: one step at a time, only if reliability clears the band line by 3+ points, and never with a `recent_late` flag. Cobalt Ridge and Ostrava sit at 87, inside the margin, so they wait.
3. Downgrade: up to two steps at once (Orchard 3 → 5).
4. Every change is a proposal. `required_role` is `CONTROLLER`, and `transition_work_item` already blocks the agent from approving its own work.
5. No duplicates: skip a client that already has an open `CREDIT_REVIEW` with the same `subject_id`.

The `CREDIT_REVIEW` work item

- `subject_id`: `tier:<client_id>:<from>-><to>`, or `review:<client_id>` for a strategic client

- `title`: for example "Orchard Treasury: tier 3 → 5"
- `amount_cents`: undisputed exposure
- `policy_ref`: pol-020 for a downgrade, pol-021 for an upgrade, pol-022 for a strategic review
- `proposal_json`: `{action, from_tier, to_tier, reliability, value, coverage_ratio, breakdown, flags, expected_settle_on}`
- `confidence`: 0.9 when nothing is near a band line, 0.7 when within 5 points
- `reasoning`: Claude's paragraph citing the numbers, or the template if Claude fails

On approval, a new `CREDIT_REVIEW` case in `executeApproved` sets `client.tier` and writes an audit event. It also adds a `precedent` row with `source = 'live'`, so the decision is remembered. Contract grace and cure days stay as signed until renewal; see the open questions. A rejection is also stored as a precedent, and the engine won't propose the same change again for 30 sim days.

Schema and reducer changes

No table changes. We add two reducers, one approval case and three policies, so the shared database isn't wiped. Only Rajeev publishes the module (`pnpm stdb:publish`), so this goes in one PR he reviews before noon.

Change	File	What it does
`upsert_credit_profile` (new)	`spacetime/src/reducers/credit.ts`	Args: client_id, value_score u8, reliability_score u8, coverage_ratio f64, coverage_detail, expected_settle_on? Roles AGENT or ADMIN. Upserts the row with `updatedAt`, and audits `CREDIT_SCORED` only when a score moves

Change	File	What it does
`upsert_exploit_flag` (new)	same file	Args match the `exploit_flag` columns. Roles AGENT or ADMIN. Inserts or updates by id, never overwrites a row a controller set to `cleared`, and audits `EXPLOIT_FLAGGED` on first insert
`CREDIT_REVIEW` case	`executeApproved` in `reducers/actions.ts`	For `action = tier_change`: check `from_tier` still matches, set `client.tier`, insert a `precedent` (`source = 'live'`), audit `TIER_CHANGED`. For `action = review`: precedent only
Rejection memory	`transitionWorkItem` in `reducers/work.ts`	When a `CREDIT_REVIEW` goes to `REJECTED`, insert a precedent with the controller's note as `override_reason`
Policies pol-020 to pol-022	`BASELINE` in `reducers/policies.ts`	pol-020 downgrade on score or exploit flag; pol-021 upgrade one step with controller approval; pol-022 strategic tiers change only by a person
Export the reducers	`spacetime/src/index.ts`	Same pattern as the other reducer files

Change	File	What it does
Bindings	`pnpm stdb:generate`	Regenerates `server/` and `web/` bindings

The agent calls the two reducers through `SpacetimeClient.call()`. It proposes `CREDIT_REVIEW` and `EXPLOIT_FLAG` items through the existing `propose_work_item`, so nothing new is needed on the proposal side.

Code layout

The scoring lives in pure functions with no network calls. The agent class only reads tables, calls those functions and writes results, so the same code runs in tests, in the 15-second loop and in replay.

Path	New or changed	Contents
`agents/src/desk_agents/credit/model.py`	New	Typed inputs (`ClientBook`: client, contract, clauses, invoices with payment dates, disputes, custody, owed) and outputs (`Scores`, `Flag`, `TierDecision`)
`agents/src/desk_agents/credit/load.py`	New	Builds a `ClientBook` per client from SQL: `client`, `contract`, `contract_clause`, `invoice`, `payment_application` joined to `cash_movement`, `dispute`, `custody_holding`, `owed_to_client`, `clock`. Also loads from a JSON fixture for tests
`agents/src/desk_agents/credit/scoring.py`	New	`reliability()`, `value()` (needs the whole book for the log scale), `coverage()`, `expected_settle_on()`, each returning its breakdown
`agents/src/desk_agents/credit/detectors.py`	New	The six detectors, each returning `(fired, measured, evidence, fix)`
`agents/src/desk_agents/credit/tiers.py`	New	Formula tier, guardrails, the decision and its reasons

Path	New or changed	Contents
`agents/src/desk_agents/credit/reasoning.py`	New	Claude call with the numbers and evidence ids, falling back to a template
`agents/src/desk_agents/agents/credit_analyst.py`	Changed	`run()`: load, score, detect, write profiles and flags, return `CREDIT_REVIEW` and `EXPLOIT_FLAG` proposals not already open
`agents/src/desk_agents/stdb.py`	Changed	`upsert_credit_profile()`, `upsert_exploit_flag()` wrappers
`agents/tests/fixtures/book.json`	New	Rajeev's dataset dumped by the script below
`agents/tests/test_credit.py`	New	Expected tiers, flags and guardrails (section 9)
`server/src/scripts/credit-fixture.ts`	New	Runs `buildDataset()` plus `platform-data` helpers and writes the fixture; `pnpm credit:fixture`
`server/src/scripts/credit-verify.ts`	New	After `pnpm sync`, reads `credit_profile`, `exploit_flag` and open `CREDIT_REVIEW` items from the live database and checks them against the expected table; `pnpm credit:verify`
`spacetime/src/reducers/credit.ts` and friends	New and changed	Section 7
`docs/credit.md`	New	This spec, short, in the style of the other docs

Add `pytest` to the agents' dev dependencies, and run the tests with `uv run pytest` in `agents/`.

Expected results and tests

On Rajeev's data the engine proposes 9 tier changes and 1 strategic review, keeps 20 clients where they are, and raises 7 flags: the six planted patterns plus Fieldstone's real slide, and no false ones. This is the answer key the tests assert. Scores may move a point

or two once payment checking has applied the March 2–3 wires; the tiers and flags must not change.

Client	Now	Proposed	Reliability	Value	Flag	Note
Aster Point Capital	1	—	94	78		
Halcyon Capital	1	review	53	98	`quarter_end`	Strategic: formula says 4, so a review goes to the controller
Redwing Liquidity	1	—	94	100		Strategic
Basalt Global Macro	2	—	92	73		Value just under 75
Bluefin Arbitrage	2	—	92	59		
Cinder Peak Partners	2	—	92	55		
Copperline Treasury	2	—	93	29		
Fieldstone Quant	2	3	77	65	`recent_late`	Missed December; caught up by lump sum on 2026-03-02
Meridian Flow Partners	2	1	93	84		
Polaris Yield Fund	2	—	90	69		
Silverline Markets	2	1	92	78		
Tidewater Asset Mgmt	2	—	93	54		
Cobalt Ridge Treasury	3	—	87	54		Inside the 3-point margin
Granite Bay Holdings	3	2	92	53		
Harbor & Pine Treasury	3	2	93	19		
Ivory Lane Capital	3	—	74	39		One 41-day miss, not a trend

Client	Now	Proposed	Reliability	Value	Flag	Note
Kestrel Quant	3	—	80	73	`recent_late`	No upgrade while getting worse
Lantern Payments	3	—	73	19		
Larkspur Treasury	3	2	92	13		
Northfold Trading	3	—	81	63		
Orchard Treasury	3	5	42	40	`dispute_delay`	Two-step downgrade
Ostrava Digital Assets	3	—	87	47		Inside the 3-point margin
Quillon Systems	3	2	93	17		
Ember Street Capital	4	—	72	52		Inside the 3-point margin
Juniper Labs	4	—	65	44	`restriction_eve`	
Monarch Rails	4	—	63	39	`pays_restricted_only`	
Saltmarsh Ventures	4	—	70	28		
Wrenfield Digital	4	5	46	24	`partial_reset`	
Tessellate Pay	5	—	54	30		
Vantage Loop	5	—	44	40		

Tests to write (`agents/tests/test_credit.py` , on the fixture):

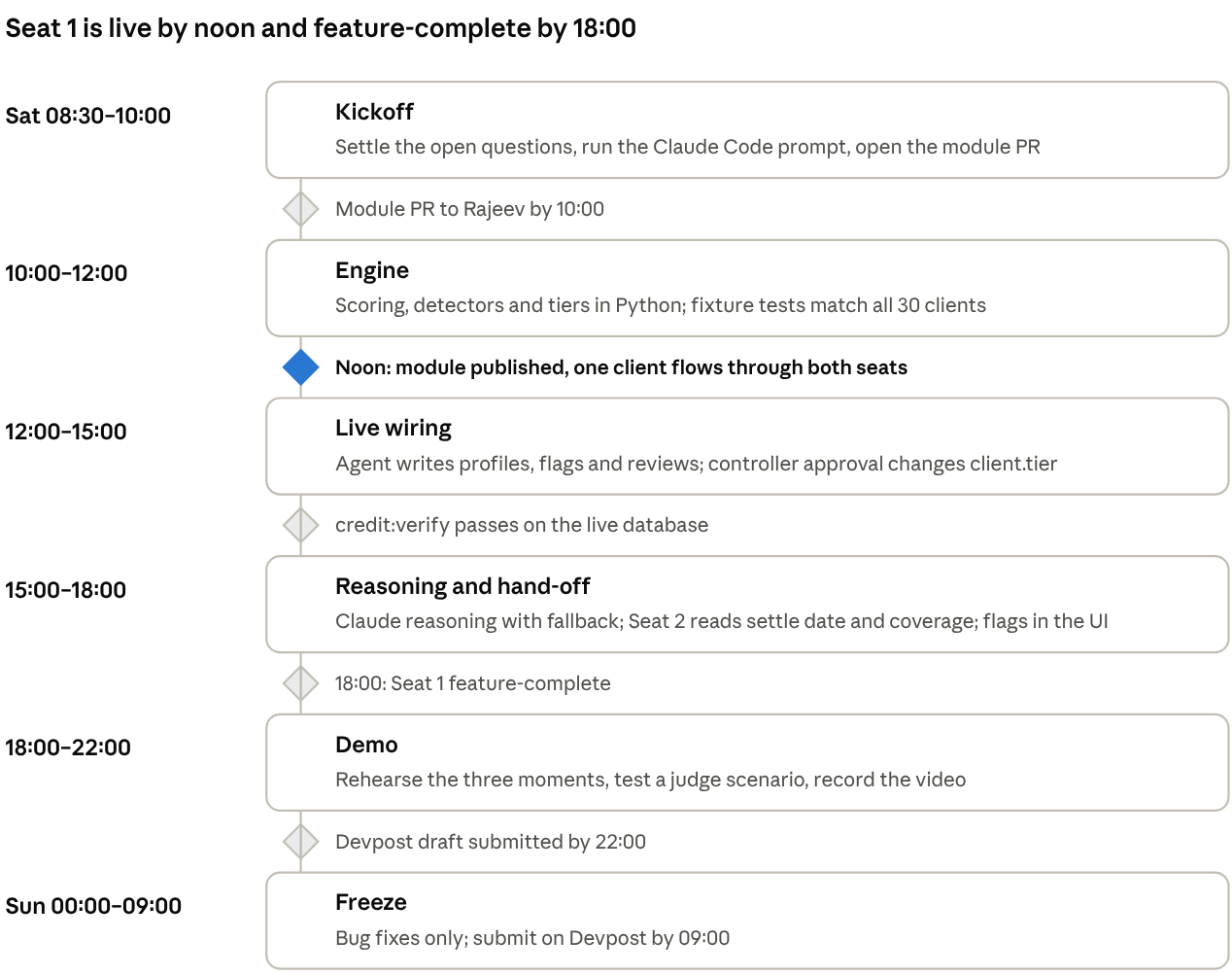
- [ ] Same input twice gives identical output
- [ ] Each of the six patterns fires on its planted client only (6 true flags, 0 false)
- [ ] Proposed tier matches the table for all 30 clients
- [ ] Halcyon and Redwing never get a `tier_change` ; Halcyon gets a `review`
- [ ] Kestrel is not upgraded while `recent_late` fires
- [ ] Cobalt Ridge, Ostrava and Ember are held by the 3-point margin
- [ ] A `credited` dispute costs nothing; a `no_change` one costs 1.5
- [ ] Nothing dated after `clock.sim_date` is read (replay at 2025-12-31 sees no January payments)

- [ ] A second `run()` with the same data proposes nothing new

Live check (`pnpm demo:reset && pnpm sync && pnpm credit:verify`): 30 `credit_profile` rows, 7 `exploit_flag` rows from the agent plus any the module raised, and 10 open `CREDIT_REVIEW` items. As the agent, approving one is denied and audited; as the controller it posts, and `client.tier` changes.

Build timeline

The noon gate is the one that matters: the module PR must be published, so Zayyan can build Seat 2 against real `client.tier` and `credit_profile` rows.



Seat 1 build timeline · 6 phases, 5 gates

If the noon gate slips, Seat 2 keeps working from the seeded tiers. Seat 1 then demos from the fixture and the replay until the module is live.

Demo moments

Three Seat 1 moments, each hitting one of Hyphenate's judging questions. Rehearse them in this order.

1. **Catches what others missed (Orchard).** It has never been past due on paper: every invoice was disputed, so it was never in default. The engine shows 22 of 22 disputes upheld, each paid within 3 days of the ruling, and proposes tier 3 → 5.
2. **Knows when to ask (Halcyon).** The platform's biggest strategic client pays in 5 lumps a year. On behaviour alone it would be tier 4. The engine doesn't touch it, and sends a review to the controller with the evidence.
3. **Limited access (any `CREDIT_REVIEW`).** As the agent, approve Meridian's upgrade: the audit log shows "no self-approval" denied. Switch to the controller and approve it: `client.tier` changes and a precedent is recorded.

Backup moment if a judge brings their own scenario: insert a new late payment for Kestrel with `/demo/inject`. Within one agent cycle (15 seconds) its reliability drops, and the reasoning cites the new invoice.

Open questions

Five decisions to settle with Rajeev and Zayyan before 10:00. The build defaults are shown, so work can start now.

- [ ] **Does a tier change touch the contract?** Default: approval sets `client.tier` now; grace and cure stay as signed until `renewal_on`. Rajeev to confirm nothing in the module re-derives contract terms from tier.
- [ ] **Juniper's planted day 20.** Under the current contract, restriction lands on day 16 (5 grace + notice + 10 cure). Default: the detector describes it as "always pays on day 20". Rajeev may move the history to day 15 to keep the "day before restriction" story.
- [ ] **Who publishes the module?** Default: Aryan opens the PR with section 7, Rajeev reviews, and publishes before 12:00.
- [ ] **Does Seat 2 read anything from Seat 1 beyond `client.tier`?** Default: Zayyan's Access Controller also reads `credit_profile.expected_settle_on` and `coverage_ratio` to pick reminder vs custody debit vs hold.
- [ ] **Agent identity.** Default: Seat 1 uses `STDB_AGENT_TOKEN` with role `AGENT`, shared with Seat 2. If Zayyan wants separate audit actors, `pnpm stdb:roles` adds a second agent identity.

Claude Code prompt

Run this from the repo root in Claude Code. It builds sections 4 to 9 end to end and stops before publishing the module, which Rajeev does. The same prompt is attached in chat as a file.

```
You are building Seat 1 (the Credit Analyst agent) inside the access-desk
repo (github.com/projects-hacks/access-desk). This is a Cal Hacks project for
the Hyphenate track. Work on a new branch `seat1-credit-analyst`. Commit in
small steps. Do not push, and do not run `pnpm stdb:publish`: Rajeev owns the
shared database and publishes the module himself.

## Step 0: read the build plan in detail

The build plan is the PDF "Seat 1 Build Plan: Credit Analyst". Save it in the
repo as `docs/seat1-build-plan.pdf` if it isn't there yet. Read the whole
thing, section by section, before writing any code:
- goal and scope
- what the repo already gives us
- architecture and data flow
- scoring spec
- exploit detectors
- tier rules, guardrails and approvals
- schema and reducer changes
- code layout
- expected results and tests
- build timeline
- demo moments
- open questions

The plan is the source of truth, and this prompt is the checklist for
building it. Before you start coding, give me a short summary of the plan and
list anything in it that no longer matches the repo (renamed files, changed
tables, new commits). Then carry on, unless a mismatch blocks the build. If
the plan and this prompt ever disagree, follow the plan and tell me. Re-read
the relevant section of the plan at the start of each part below.

## Read the repo

README.md, docs/schema.md, docs/actions.md, docs/history.md,
docs/payments.md, spacetime/src/tables/*.ts,
spacetime/src/reducers/{work,actions,policies}.ts, spacetime/src/context.ts,
spacetime/src/roles.ts, spacetime/src/index.ts,
```

```
server/src/dataset/{clients,history,scenario,platform-data,build}.ts, and
everything in agents/src/desk_agents/. Match the existing code style:
TypeScript reducers per area, audit on every state change, Python with
pydantic and httpx, ruff line length 100.

## What Seat 1 does

For every client it computes three scores (reliability, value, coverage) plus
a predicted pay date, and writes them to `credit_profile`. It runs six
exploit detectors and writes `exploit_flag` rows. It proposes tier changes on
Rajeev's existing 1–5 scale (1 = best) as `CREDIT_REVIEW` work items, which
only a CONTROLLER can approve.

Seat 1 never restricts levers, holds cash, debits custody, approves its own
work, changes a strategic client's tier or edits contract terms. The scoring
is deterministic Python. Claude only writes the reasoning text, with a
template fallback. "Today" is always `clock.sim_date` (2026-03-03 after a
demo reset), never the wall clock. Nothing dated after today may be read, so
replays stay honest.

Note: the existing `client_credit` table means unapplied cash. Seat 1 never
writes it.

## Part 1: SpacetimeDB module (no table changes)

1. New file `spacetime/src/reducers/credit.ts`, exported from
`spacetime/src/index.ts` like the other reducer files:
   - `upsert_credit_profile(client_id, value_score u8, reliability_score u8,
coverage_ratio f64, coverage_detail string, expected_settle_on
option<string>)`. Use `requireRole(ctx, ['AGENT', 'ADMIN'])`. Insert or
update by client_id, set `updatedAt = ctx.timestamp`, and audit
`CREDIT_SCORED` (subjectType `credit_profile`) only when a score changed.
   - `upsert_exploit_flag(id, client_id, pattern, score f32, status, months
u8, evidence_json, proposed_fix)`, roles AGENT or ADMIN. Insert or update by
id. If the existing row has status `cleared`, keep `cleared`. Audit
`EXPLOIT_FLAGGED` on first insert only.
2. In `executeApproved` (reducers/actions.ts), add `case 'CREDIT_REVIEW'`.
Parse `proposalJson`:
   - `action === 'tier_change'`: refuse quietly with an audit row if
`client.tier !== from_tier` (stale). Otherwise update `client.tier` to
`to_tier` and audit `TIER_CHANGED` with from/to.
   - Both `tier_change` and `review`: insert a `precedent` row: id `PRC-
L<workItemId>`, client_id, work_item_id, decided_on = sim date, situation =
```

```
item title, proposed = `tier X -> Y` or `review`, decision `Approved`,
override_reason '', outcome '', policy_ref, source `live`.
   - Contract grace/cure days are NOT changed (they change at renewal).
3. In `transitionWorkItem` (reducers/work.ts): when a `CREDIT_REVIEW` moves
to `REJECTED`, insert the same kind of precedent with decision `Rejected` and
override_reason = the note.
4. Add to `BASELINE` in reducers/policies.ts:
   - pol-020 "Downgrade a client's tier on score or exploit flag, with
controller approval"
   - pol-021 "Upgrade a client's tier one step at a time, with controller
approval"
   - pol-022 "Strategic clients' tiers change only by a person; the agent may
request a review"
5. Run `pnpm stdb:generate` (bindings) and `pnpm typecheck`. If the spacetime
CLI isn't available, write the code and say so in the report.

## Part 2: Python package `agents/src/desk_agents/credit/`

Keep the scoring pure: no network calls in model/scoring/detectors/tiers.

- `model.py`: pydantic or dataclass types. `ClientBook` holds client (id,
name, segment, tier, strategic, renewal_on), contract (terms/grace/cure days,
cross_default), clauses, invoices (id, product, period, issued_on, due_on,
amount_cents, paid_cents, paid_on, status, payments = list of (value_on,
amount_cents)), disputes (invoice_id, opened_on, resolved_on, resolution,
status), custody holdings (asset, value_cents), owed_to_client total, today.
Outputs: `Scores` (with breakdown), `Flag`, `TierDecision`.
- `load.py`: `load_books(db) -> list[ClientBook]` reads whole tables with
`db.sql("SELECT * FROM <table>")`: client, contract, contract_clause,
invoice, payment_application, cash_movement, dispute, custody_holding,
owed_to_client, clock. Join in Python; don't rely on SQL joins. Payments per
invoice = payment_application rows where `reversed` is false, joined to
`cash_movement.value_on` by movement_id. Drop anything dated after today.
Also add `load_books_from_json(path)` and a `python -m desk_agents.credit
dump <path>` command that writes the live books to JSON.
- `scoring.py`:
  - Invoices scored: due_on < today, status not void, not under an open
dispute (status `under_review`). Paid: lag = paid_on − due_on. Open: lag =
today − due_on. dpd = max(0, lag). age = today − due_on. weight = 0.5 ** (age
/ 90).
  - Reliability: start at 100 and subtract:
    - lateness = min(35, 0.8 × weighted mean dpd)
    - frequency = 20 × weighted share with dpd > 10
```

```
    - severity = 15 if the max dpd among invoices with age ⩽ 180 is > 60,
else 8 if > 30, else 0
    - pieces = min(10, 4 × weighted count of invoices with 2+ payments)
    - disputes = min(15, 1.5 × count of the client's disputes resolved
`no_change`; `credited` costs 0)
    - raw = max(0, 100 − sum). m = number of distinct periods scored.
reliability = (m × raw + 3 × 70) / (m + 3).
  - Value: t12 = sum of amount_cents of invoices issued in the 365 days
before today. value = 75 × (ln(t12) − min ln) / (max ln − min ln) across all
clients + 25 × (products billed / 4).
  - Coverage: liquid = (USD custody value if a contract_clause of kind
`debit` is allowed, else 0) + owed_to_client. exposure = unpaid (amount −
paid) on invoices not paid/void/disputed. coverage_ratio = liquid / exposure
(0 if exposure is 0). coverage_detail is a short sentence: "Debitable $X
under §6.3 (or 'no §6.3 debit clause'); holdable custody $Y under §7.2 = Nx
exposure; owed to client $Z."
  - Expected settle: the weighted median of lag over paid scored invoices,
using the same weights. expected_settle_on = due_on of the oldest open
undisputed invoice + that lag, or today if earlier; None if nothing is open.
- `detectors.py`: each returns fired, measured (a string), evidence (a list
of {invoice_id, due_on, paid_on, dpd, note}), months covered, score 0–1,
proposed_fix. restriction_day = grace + 1 + cure.
  1. `restriction_eve`: take paid invoices with exactly 1 payment in the last
8 distinct periods. Fire if there are 6 or more, the most common lag covers ⩾
85% of them, and that lag > grace.
  2. `partial_reset`: invoices with 2+ payments where the first payment < 60%
of the amount and the last payment's value_on − due_on > restriction_day.
Fire if 2 or more.
  3. `pays_restricted_only`: median lag per product over paid invoices. Fire
if there are 2+ products, max − min ⩾ 15, and max > grace.
  4. `quarter_end`: distinct payment dates in the 365 days before today. Fire
if the count is between 1 and 5 and (distinct periods paid in that window /
payment dates) ⩾ 2.
  5. `recent_late`: per period, the max dpd among paid invoices, ordered by
period. last3 = the last 3 periods, prior = the rest. Fire if 2 or more of
last3 are > 5 and mean(last3) − mean(prior) ⩾ 7.
  6. `dispute_delay`: fire if (distinct periods with a dispute / periods
scored) ⩾ 0.5, share `no_change` ⩾ 0.8, and share of disputed invoices paid
within 3 days after resolved_on ⩾ 0.8.
- `tiers.py`:
  - Base tier from reliability: ⩾ 85 → 2, ⩾ 70 → 3, ⩾ 55 → 4, else 5. If
value ⩾ 75, base − 1 (min 1). Tier 1 requires reliability ⩾ 85. Any flag →
max(tier, 3). That is the formula tier.
```

```
  - Strategic: never a tier change. If formula tier > current, return action
`review`.
  - Upgrade (formula < current): blocked if `recent_late` fired. Blocked if
reliability < (lower bound of the base band before the value bump) + 3.
Otherwise propose current − 1 (one step).
  - Downgrade (formula > current): propose current + min(2, formula −
current).
  - confidence = 0.7 if reliability is within 5 of any band line (55, 70,
85), else 0.9.
  - policy_ref: pol-020 downgrade, pol-021 upgrade, pol-022 review.
  - Return the reasons for every block, for the reasoning text.
- `reasoning.py`: `explain(decision, scores, flags) -> str`. If
`settings.anthropic_api_key` is set, call Claude (`settings.anthropic_model`)
with only the numbers, the breakdown and the evidence invoice ids. Ask for at
most 120 words, citing ids, with no new numbers. Use a 20-second timeout. On
any error or with no key, return a template built from the same numbers.
Tests always use the template.

## Part 3: agent and client wiring

- `stdb.py`: add `upsert_credit_profile(...)` and `upsert_exploit_flag(...)`
wrappers around `call()`.
- `agents/credit_analyst.py`, `run()`:
  1. Load books.
  2. Score all clients.
  3. Call `upsert_credit_profile` for each (scores rounded to int 0–100).
  4. Run the detectors. Call `upsert_exploit_flag` with id `<pattern>:
<client_id>`, status `watch` for strategic clients and `flagged` otherwise,
evidence_json = JSON of the evidence.
  5. Decide tiers.
  6. Return `Proposal`s (models.py) for new items only:
     - `CREDIT_REVIEW` with subject_id `tier:<client_id>:<from>-><to>` or
`review:<client_id>`; title like "Orchard Treasury: tier 3 → 5" or "Halcyon
Capital: review strategic tier"; amount_cents = exposure; proposal = {action,
from_tier, to_tier, reliability, value, coverage_ratio, breakdown, flags,
expected_settle_on}; required_role CONTROLLER.
     - One `EXPLOIT_FLAG` per new flag, subject_id `exploit:<pattern>:
<client_id>`, required_role CONTROLLER.
  - Skip any subject_id that already has a work item in PROPOSED, NEEDS_INFO,
APPROVED or POSTED, or a live precedent from a rejection less than 30 sim
days ago. A second run with unchanged data must propose nothing.

## Part 4: fixture, tests, verify, docs
```

- `server/src/scripts/credit-fixture.ts` plus a `pnpm credit:fixture` script. It builds books from `buildDataset()` and the platform-data helpers (contractFor, clausesFor, holdingsFor, owedToClient, renewalOn), in the same JSON shape `load_books_from_json` reads, and writes `agents/tests/fixtures/book.json`. Payments for history invoices come from `history.cash` (value date + amount per ref). Bank-window movements are unapplied, except those `buildDataset` already marked paid.
- Add pytest to the agents dev dependencies. Write `agents/tests/test_credit.py` on the fixture, with this answer key (tier now → expected):
  - ORG-90101 Halcyon 1 → review [quarter_end]
  - ORG-90107 Redwing 1 → stay
  - ORG-90118 Aster Point 1 → stay
  - ORG-90142 Meridian Flow 2 → 1
  - ORG-90164 Silverline 2 → 1
  - ORG-90176 Northfold 3 → stay
  - ORG-90229 Fieldstone 2 → 3 (recent_late fires only once the 2026-03-02 lump sum is applied, so don't assert the flag on the fixture)
  - ORG-90233 Bluefin 2 → stay
  - ORG-90244 Basalt 2 → stay
  - ORG-90251 Cinder Peak 2 → stay
  - ORG-90287 Tidewater 2 → stay
  - ORG-90302 Polaris 2 → stay
  - ORG-90355 Cobalt Ridge 3 → stay (held by 3-point margin)
  - ORG-90366 Copperline 2 → stay
  - ORG-90378 Harbor & Pine 3 → 2
  - ORG-90391 Granite Bay 3 → 2
  - ORG-90399 Larkspur 3 → 2
  - ORG-90412 Kestrel 3 → stay [recent_late]
  - ORG-90431 Ostrava 3 → stay (held by margin)
  - ORG-90455 Ember Street 4 → stay (held by margin)
  - ORG-90463 Ivory Lane 3 → stay (no flag)
  - ORG-90518 Orchard 3 → 5 [dispute_delay]
  - ORG-90612 Saltmarsh 4 → stay
  - ORG-90677 Juniper 4 → stay [restriction_eve]
  - ORG-90689 Quillon 3 → 2
  - ORG-90731 Lantern 3 → stay
  - ORG-90744 Wrenfield 4 → 5 [partial_reset]
  - ORG-90758 Monarch 4 → stay [pays_restricted_only]
  - ORG-90802 Vantage Loop 5 → stay
  - ORG-90816 Tessellate 5 → stay

```
  Also test:
  - determinism (same input twice gives identical output)
  - exactly the flags above on the fixture (no others)
  - no tier_change for strategic clients
  - Kestrel not upgraded
  - `credited` disputes cost 0
  - a replay at today = 2025-12-31 reads no 2026 payments
  - idempotent proposals (a second run with the open items present returns
[])

  Expected scores, approximately: Halcyon reliability 53 / value 98; Orchard
42 / 40; Juniper 65 / 44; Kestrel 80 / 73; Meridian 93 / 84. If a tier
differs from the key, show me the breakdown before changing any weight. Don't
tune weights silently.
- `server/src/scripts/credit-verify.ts` plus `pnpm credit:verify`. Against
the live database, check:
  - 30 credit_profile rows
  - the agent's exploit_flag rows match the key (plus Fieldstone
`recent_late` once payment checking has applied its lump sum)
  - one open CREDIT_REVIEW per expected change or review (10)
  - Print a table.
- `docs/credit.md`: a short spec in the style of the other docs (scores,
detectors, tiers, guardrails, reducers, how to verify). Link it from the
README docs line.

## Done when

`pnpm typecheck` and `pnpm test` pass, and `cd agents && uv run pytest && uv
run ruff check` pass. Go back through the plan's "Expected results and tests"
section and tick off every item. Then report:
- what you built
- the test output
- the 30-client table (now, formula tier, proposed, reliability, value,
coverage, flags)
- anything you couldn't do, for example publishing or a live verify that
needs Rajeev to publish the module first

Don't run destructive commands on the shared database other than `pnpm
demo:reset`, and only if I confirm.
```